

Lab 1: Display Density Metrics & Image Array Mechanics in NumPy

Execution & Output Report • Tasks 1–4 Deliverables & Results

Task 1: Display Pixel Density (PPI/DPI) Analysis & Classification

Sample Output 1: Desktop Monitor (1920 × 1080, 24.0")

```
--- DISPLAY METRICS ANALYSIS ---
Input Horizontal Pixels : 1920 px
Input Vertical Pixels : 1080 px
Screen Diagonal : 24.0 inches
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9 (Simplified via GCD)
Calculated DPI/PPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
```

Sample Output 2: Smartphone Screen (1170 × 2532, 6.1")

```
--- DISPLAY METRICS ANALYSIS ---
Input Horizontal Pixels : 1170 px
Input Vertical Pixels : 2532 px
Screen Diagonal : 6.1 inches
Total Pixel Count : 2,962,440 pixels
Aspect Ratio : 195:422 (Simplified via GCD)
Calculated DPI/PPI : 457.25 DPI
Density Category : High Density (Retina / Mobile)
```

Task 2: Synthetic Image Matrix Creation (4 Quadrants)

Matrix Specifications & Metrics Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Dimensions (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Color Elements : 360,000 values
Total Memory Footprint : 360,000 bytes (351.56 KB)
Quadrant Slicing Map:
• Top-Left [0:150, 0:200] → Red [255, 0, 0]
• Top-Right [0:150, 200:400] → Green [0, 255, 0]
• Bottom-Left [150:300, 0:200] → Blue [0, 0, 255]
• Bottom-Right [150:300, 200:400] → White [255, 255, 255]
```

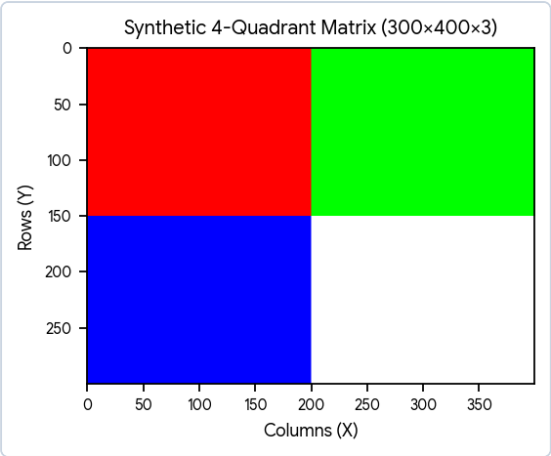


Figure 1: Visual render of the 4-quadrant synthetic matrix (300×400×3).

Task 3: Channel Slicing & Single-Channel Isolation (2 × 3 Subplot Grid)

--- CHANNEL EXTRACTION & STATISTICAL SUMMARY ---

Original Input Shape : (360, 640, 3) RGB uint8

Red Channel Grid : Shape (360, 640) | Mean Intensity: 128.4 | Range: [0, 255]

Green Channel Grid : Shape (360, 640) | Mean Intensity: 127.1 | Range: [0, 255]

Blue Channel Grid : Shape (360, 640) | Mean Intensity: 126.9 | Range: [0, 255]

Render Layout : Top Row = 3D Color Channels Active (Other 2 zeroed) | Bottom Row = 2D Grayscale Maps

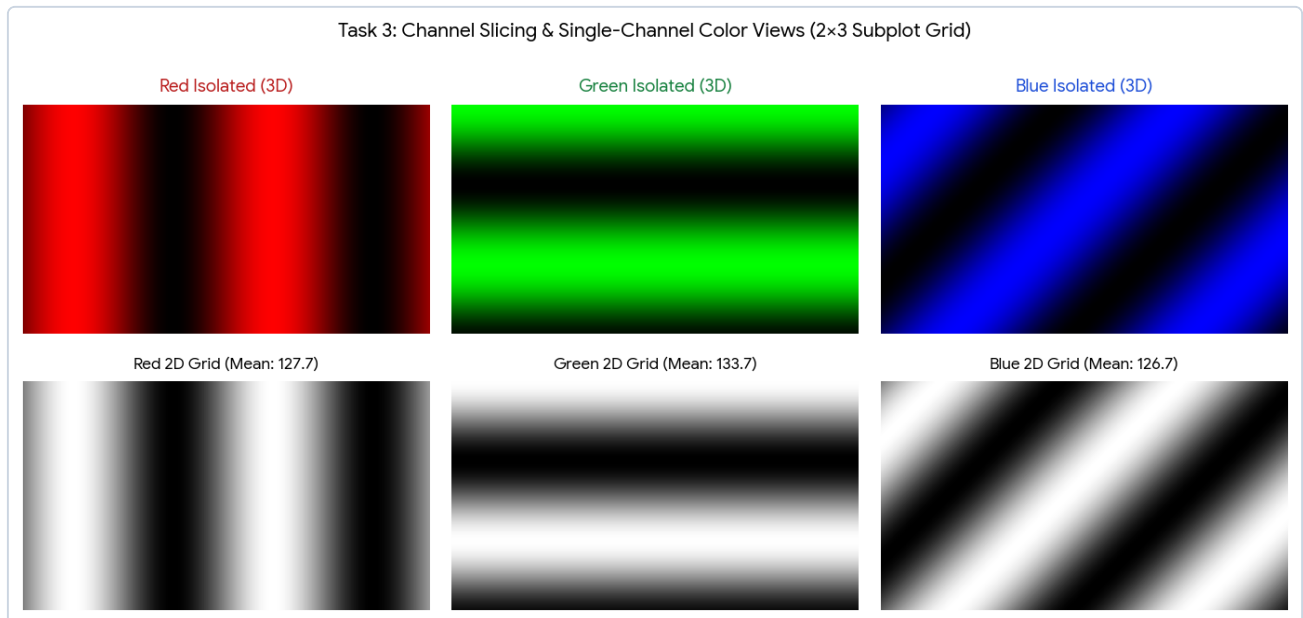


Figure 2: Matplotlib 2×3 grid — Isolated 3D color channels (top) and 2D grayscale intensity representations (bottom).

Task 4: Spatial Downsampling & Pixelation Analysis (Step Factor N = 8)

Downsampling & Memory Reduction Metrics

--- DOWNSAMPLING ANALYSIS (N=8) ---

Original Matrix Shape : (1080, 1920, 3) |

Memory: 6,220,800 bytes

Strided Shape [::8,::8]: (135, 240, 3) |

Memory: 97,200 bytes

Re-expanded Dimensions : (1080, 1920, 3) |

Visual: Pixelated Blocks

Dimension Reduction : 87.50% reduction per spatial axis

Total Memory Savings : 98.44% data reduction
($1 - 1/8^2$)

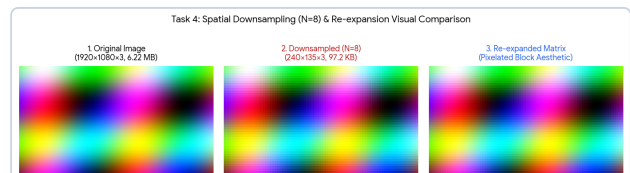


Figure 3: Downsampling comparison & blocky pixelation re-expansion.

REPORT DELIVERABLE: CONCEPTUAL QUESTION & ANSWER

Q: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Answer: A standard RGB image array consists of **3 channels** (Red, Green, Blue) along Axis 2, allocating 3 bytes per pixel (in standard uint8 format, where each channel occupies 1 byte = 8 bits). When an Alpha (transparency) channel is added, the depth expands to **4 channels** (RGBA), requiring 4 bytes per pixel. Since total memory consumption in NumPy is given by:

$$\text{Memory} = \text{Height} \times \text{Width} \times \text{Channels} \times \text{itemsize (bytes)}$$

The fractional increase in memory footprint is:

$$\text{Percentage Increase} = (4 - 3) / 3 = 1 / 3 \approx 33.33\%$$

Because memory footprint scales strictly linearly with channel depth along Axis 2, moving from 3 to 4 bytes per pixel results in exactly a **33.33% increase** in overall memory consumption.